

Multi-Task Learning in CNNs for Classification, Segmentation, and Detection

Overview of Multi-Task Learning in CNNs

Multi-task learning (MTL) refers to training a single model (e.g. a CNN) to perform multiple tasks simultaneously – in this context, image classification, semantic segmentation, and object detection all in one network. Typically, the model has a **shared backbone** (convolutional feature extractor) and separate **task-specific heads** for each task (e.g. one head outputs class labels, another produces pixel-wise segmentation masks, another predicts bounding boxes). By optimizing multiple loss functions jointly, the model learns **shared representations** that serve all tasks ¹ ². This unified approach contrasts with training three independent models for each task. Below, we examine the key advantages and disadvantages of such multi-task training, along with when it is beneficial or harmful.

Advantages of Multi-Task Training

- **Improved Generalization & Reduced Overfitting:** Sharing representations across tasks provides an inductive bias that helps the model generalize better on each individual task ¹. The model is encouraged to learn features that explain more than one task, acting as a form of regularization. In fact, using hard parameter sharing, the risk of overfitting the shared layers drops significantly as the number of tasks increases ³. Empirically, multi-task CNNs often exhibit lower overfitting and better overall accuracy by leveraging the additional training signals from related tasks ⁴. As Caruana famously stated, “*MTL improves generalization by leveraging the domain-specific information contained in the training signals of related tasks.*” ¹.
- **Data Efficiency:** A multi-task model effectively “sees” more data variation by learning from multiple task datasets. This acts as an **implicit data augmentation** – different tasks have different noise patterns, so the shared model must learn features that generalize across these variations ⁵. The result is higher data efficiency: the model can learn robust features from fewer task-specific samples than a single-task model might need ⁴. In other words, the tasks serve as additional context for each other, which can accelerate learning and improve sample efficiency (leveraging the same images for multiple labels). Studies have noted that this approach “*increases data efficiency [and] decreases overfitting by exploiting shared features from multiple tasks*” ⁴.
- **Computational and Memory Efficiency (Inference):** Deploying one CNN for multiple tasks is more resource-efficient than maintaining separate models. A single forward pass yields predictions for classification, detection, and segmentation together, saving computation and runtime. For example, the **MultiNet** architecture (for road scene understanding) uses one shared encoder for classification, object detection, and road segmentation; it achieved real-time performance – **under 100 ms for all three tasks combined** ⁶. This unified model also won the KITTI road segmentation benchmark while performing detection and classification concurrently ⁶, demonstrating that multi-task models can be both accurate and efficient. In general, one multi-task network uses far fewer

parameters than three separate networks, saving storage and making deployment easier ⁷. This is especially important in real-world applications (autonomous vehicles, mobile devices) where **computational resources and memory are limited** ⁸.

- **Leveraging Shared Features & Context:** By learning tasks together, the CNN can develop **shared low-level and mid-level features** (edges, textures, object parts) useful for all tasks. This can provide complementary signals – for instance, segmentation can force the network to learn precise spatial features, which may also improve the localization accuracy of object detection. Likewise, detection and classification heads provide global context that can help disambiguate segmentation classes. In multi-task autonomous driving models, researchers find that **joint learning of detection and segmentation yields better scene understanding** than separate models ⁹ ⁶. The tasks essentially act as *auxiliary training objectives* that guide the network's attention: if one task finds a certain feature useful, the shared encoder will emphasize it, benefiting the other tasks ("eavesdropping" on features that are easier to learn in one task ¹⁰). This synergy can sometimes **improve each task's performance** beyond what separate training would achieve. For example, adding a segmentation branch has been shown to slightly improve object detection accuracy by learning richer features, and some multi-task models even achieve **state-of-the-art results on individual tasks** when designed well ⁶ ¹¹.
- **Overall Training Efficiency:** When considering the big picture (all tasks), a multi-task approach can be more efficient to train and maintain. Rather than training and tuning three separate CNNs, we train one unified model. This can shorten development cycles and, if tasks are learned in parallel, potentially reduce total training time compared to sequential training of multiple models. In some cases, **training signals from related tasks can speed up convergence** – the model receives more gradient feedback per input (summing multiple loss terms), which can accelerate learning of shared features ⁴. One study noted that multi-task models "*could improve learning speed by leveraging contextual information*," thereby partially alleviating the need for extremely large datasets or prolonged training for each individual task ⁴. In practice, the efficiency gain in training is nuanced (as discussed below), but the ability to train one network for many objectives is a clear practical advantage.

Disadvantages and Challenges

- **Gradient Conflict and Negative Transfer:** A core difficulty in multi-task CNNs is that the different task objectives might **conflict** with each other. If tasks are not highly related or have competing needs, their gradient updates can point in opposite directions in parameter space ¹². This means improving the network for one task might *hurt* performance on another. Simply averaging such gradients leads to a compromise where **neither task improves as well as it would in isolation** ¹². This phenomenon is known as **negative transfer** ¹³. For example, a backbone layer might face opposing pressures: the classification head might prefer invariant, abstract features, while the segmentation head demands fine-grained spatial detail – without careful design, one task's learning could deteriorate the other's. This gradient interference makes optimization tricky. Research has proposed solutions (e.g. gradient surgery, task weighting schemes) to **mitigate gradient conflicts**, but they add complexity to the training process ¹⁴ ¹⁵.
- **Optimization Difficulty (Loss Balancing):** Training multiple tasks together introduces extra hyperparameters and design choices. One must decide **how to weight each task's loss** in the joint

objective. An imbalance can cause the model to prioritize one task at the expense of others. For instance, if object detection loss dominates, the network may devote most capacity to detection while barely improving segmentation (or vice versa). Selecting appropriate loss weights or schedules is non-trivial and often requires experimentation ¹⁵. Likewise, **dataset imbalance** complicates training: if one task has far more examples (or easier labels) than another, the model will see that task more frequently and may “ignore” the smaller task ¹⁶ ¹⁷. Ensuring each task gets learned features requires techniques like resampling, task-specific batch sampling, or dynamic weight tuning ¹⁸ ¹⁷. Overall, the **optimization landscape is more complex** – instead of a single loss surface, the model must navigate a multi-objective loss, which can be harder to converge and may require special strategies (adaptive weight methods like GradNorm, uncertainty weighting, etc.) ¹⁹.

- **Increased Training Time and Complexity:** Joint training of three tasks is computationally heavier than training for one task. Each batch update involves computing multiple forward passes (one per task head) and backpropagating multiple loss gradients. As a result, **training a multi-task CNN demands more compute and often more training time than a single-task model** ²⁰. It’s been analogized to “*running several marathons instead of one*” in terms of training effort ²⁰. Efficient parallelization or multi-GPU training can help, but the fact remains that the model is solving more problems at once. In scenarios where you only need one task, this extra cost may not be justified. It’s also harder to debug or tune a multi-task model – improvements or bugs in one task can have ripple effects on others, making the training process more **complex and fragile**. In summary, while a multi-task model might ultimately be more resource-efficient **at inference**, during training it can be slower and more memory-intensive than a single-task training run (though still usually cheaper than training several separate models back-to-back if all tasks are required).
- **Potential Performance Trade-offs:** Unless tasks are truly synergistic, a jack-of-all-trades model can be a master of none. In some cases, a multi-task CNN might not reach the same peak accuracy on each individual task as dedicated single-task models would. The shared capacity forces a trade-off: the network allocates finite neurons to multiple objectives. If one task requires very task-specific features or a very different network depth/structure than the others, a unified model might **underperform specialized models**. Empirical benchmarks have shown that “*grouping some tasks hurts overall model performance*” when those tasks have low affinity ²¹. For example, if one tried to train a CNN on both image recognition and pixel-wise segmentation of unrelated attributes, the conflicting goals could lead to suboptimal results on both (this is another angle of negative transfer). There is also a risk that the “**easier**” task or **larger dataset can overwhelm training**, causing the model to saturate on that task and under-fit the harder task. Researchers emphasize that MTL is only *effective when the tasks are sufficiently related* and share common structure; otherwise, the multi-task model can actually perform worse due to noise from irrelevant objectives ²¹ ²².
- **Design and Maintenance Complexity:** Building a multi-task CNN requires careful architectural design. One must decide what layers to share and what to keep task-specific. If tasks demand different receptive field sizes or feature resolutions (e.g. segmentation benefits from high-resolution feature maps, whereas classification might be fine with pooled features), a naïve one-architecture-fits-all approach can hurt one task. Techniques like *soft parameter sharing* or custom head architectures may be needed to accommodate differences ²³ ²⁴. This added design complexity is a disadvantage in terms of development time. Moreover, debugging a multi-task model is harder – an error in the shared layers affects all outputs. Future updates to one task (say, adding new classes to detection) must be integrated into a single model without breaking others. This tight coupling

means **less flexibility** compared to separate models. In some cases, maintaining separate specialized models might be simpler, especially if tasks evolve independently.

When Multi-Task Learning Helps vs. Hurts

When MTL is Beneficial: Multi-task training shines when the tasks are **naturally related and complementary**. If tasks share substantial visual information, a common feature extractor can learn a richer, more general representation. Classic examples include joint object detection and semantic segmentation in autonomous driving: both require understanding object boundaries and categories, so sharing features is efficient and can improve accuracy ⁹ ²⁵. Indeed, many modern autonomous driving models use a multi-task CNN to simultaneously identify objects, free space (road segmentation), lane markings, etc., enabling real-time perception with one network ⁶ ²⁶. In such cases, **multi-task models often achieve comparable performance to single-task models while being far more efficient**. For instance, the YOLOP/YOLOPv3 framework handles vehicle detection, drivable-area segmentation, and lane detection together; **it achieved state-of-the-art metrics in all three tasks at once** (e.g. 84.3% mAP50 for detection and 93.2% mIoU for segmentation) while maintaining real-time speeds ¹¹. Another scenario where **MTL helps is when one task has limited data** – a related auxiliary task can act as a form of inductive transfer, improving the low-data task's generalization. Researchers have found that adding auxiliary objectives (like depth prediction or edge detection) can boost a primary task (like classification or segmentation) by providing additional supervision signal ²⁷ ²⁸. In summary, **MTL is most beneficial when tasks have high affinity (shared structure or features) and when efficiency is critical (one model doing multiple jobs)**. It also aligns with how humans learn – we transfer knowledge between related skills, which often accelerates learning of each.

When MTL Can Be Harmful: Multi-task learning is **counterproductive when tasks are unrelated or adversarial** to each other. If there is little overlap in the type of features or information needed, a shared model will struggle to serve both. In extreme cases (e.g. training a CNN on both image segmentation and a totally different task like audio classification), a single network is forced to split its capacity, essentially learning disparate functions under one roof – **the result is likely worse than two separate specialized models**. Even in more subtle cases, if one task's signal is essentially noise for another, the shared layers may become confused. This manifests as the earlier mentioned negative transfer ¹³. Practically, if adding a new task causes your original task performance to drop significantly, it's a sign that the tasks might not be compatible in a single model (or that your weighting is off). **Competing objectives** are another red flag: for instance, a classification head pushing for rotation-invariant features versus a localization task needing orientation-specific cues could conflict. Additionally, when the **task data scales are massively imbalanced**, multi-task training can be detrimental without careful handling. A model might “forget” the small task when most batches are dominated by the large-task data ²⁹ ³⁰. Lastly, if the highest possible performance on each individual task is the primary goal (e.g. in competitive benchmarks), a multi-task model might not reach the absolute peak accuracy that separate, fully-tuned models could – due to the necessity of compromise. In such cases, using separate models or a sequential two-step approach (where one task's output feeds into another, rather than joint training) could be preferable.

In conclusion, multi-task learning offers a powerful paradigm to train CNNs that **jointly handle classification, segmentation, detection** and more. Its advantages include better generalization through shared learning and greater computational efficiency in deployment ⁷ ⁶. However, these come at the cost of a more complex training process, potential gradient interference, and the need for tasks to be truly related to avoid negative transfer ¹³ ²². When tasks are well-aligned, a multi-task CNN can provide an

elegant one-model solution (as seen in many modern vision applications). But when tasks conflict or require undivided focus, forcing them together can harm performance. The decision to use multi-task training should thus weigh these trade-offs: if the tasks have **strong synergy or strict efficiency requirements**, a multi-task approach is likely beneficial; if they diverge greatly or peak accuracy is paramount, a single-task (or multi-model) approach might be safer. Ongoing research continues to refine MTL techniques (e.g. better loss balancing, task grouping algorithms ²¹ ³¹) to expand the range of scenarios where multi-task learning can be applied effectively.

Sources: Multi-task learning overview and theory ¹ ³ ⁵ ⁷ ; advantages in data efficiency and generalization ² ⁴ ; efficiency and use-case examples ⁶ ¹¹ ; challenges with negative transfer and optimization ¹² ³² ; discussion of task affinity vs. conflict ²¹ ¹³ .

¹ ³ ⁵ ¹⁰ ²² ²⁷ An Overview of Multi-Task Learning for Deep Learning

<https://www.ruder.io/multi-task/>

² ⁴ A Multi-Task Convolutional Neural Network for Semantic Segmentation and Event Detection in Laparoscopic Surgery - PMC

<https://pmc.ncbi.nlm.nih.gov/articles/PMC10054284/>

⁶ [1612.07695] MultiNet: Real-time Joint Semantic Reasoning for Autonomous Driving

<https://arxiv.org/abs/1612.07695>

⁷ ⁸ ¹² ¹³ ¹⁴ ¹⁵ ¹⁷ ²¹ ²³ ²⁴ ²⁵ ³⁰ ³¹ ³² Multi-Task Learning in ML: Optimization & Use Cases [Overview]

<https://www.v7labs.com/blog/multi-task-learning-guide>

⁹ ¹⁹ ²⁸ ml4ad.github.io

<https://ml4ad.github.io/files/papers/Boosting%20semantic%20segmentation%20with%20multi-task%20self-supervised%20learning%20for%20autonomous%20driving%20applications.pdf>

¹¹ ²⁶ Multi-Task Visual Perception for Object Detection and Semantic Segmentation in Intelligent Driving

<https://www.mdpi.com/2072-4292/16/10/1774>

¹⁶ ¹⁸ ²⁹ Implementing Multi-Task Learning For Classification and Segmentation - vision - PyTorch Forums

<https://discuss.pytorch.org/t/implementing-multi-task-learning-for-classification-and-segmentation/148385>

²⁰ Multi-Task Learning: Enhancing Model Efficiency and Generalization | by Zhong Hong | Medium

<https://medium.com/@zhonghong9998/multi-task-learning-enhancing-model-efficiency-and-generalization-4d6f5ffd2fa7>